

GRÃO E ORIGEM

Plano técnico de frete e pagamentos

Implementação em sandbox com Melhor Envio e Asaas

Frete	Melhor Envio com conta central da plataforma
Pagamento	Asaas com cobrança única por pedido
Operação	Subpedido e envio independentes por produtor
Repasse	Pix manual após postagem oficialmente confirmada

Versão 1.0 • 24 de agosto de 2026

Sumário

Estrutura do documento

- 01 Objetivo
- 02 Decisões fechadas
- 03 Situação atual do repositório
- 04 Arquitetura alvo
- 05 Modelo de dados proposto
- 06 Estados e transições
- 07 Contratos HTTP do produto
- 08 Sequência de implementação
- 09 Roteiro de homologação
- 10 Guarda-corpos técnicos
- 11 Atualizações importantes da documentação oficial
- 12 Dependências externas antes do primeiro teste integrado
- 13 Referências oficiais validadas

Objetivo

Implementar o fluxo de compra multi-produtor aprovado nos documentos de produto:

- Melhor Envio para cotação, compra, geração de etiqueta e rastreio.
- Asaas para uma cobrança única por pedido, via Pix ou cartão de crédito.
- Sem split, escrow ou conta financeira por produtor no MVP.
- Um subpedido e um fluxo logístico independente por produtor.
- Repasse manual por Pix, elegível somente após `order.posted` validado.

Este plano mantém o monólito Node.js/Express, Prisma/PostgreSQL e React existentes, mas separa as integrações externas por módulos e adaptadores.

Decisões fechadas

Tema	Decisão do MVP
Conta logística	Conta central da Grão e Origem no Melhor Envio
Origem	Endereço logístico de cada produtor
Carrinho multi-produtor	Agrupar itens e gerar um <code>SellerOrder</code> por produtor
Frete mostrado	No máximo Econômico, Rápido e, futuramente, Recomendado
Pagamento do comprador	Uma cobrança total no Asaas
Meios no lançamento	Pix e cartão de crédito
Repasse	Manual por Pix; automação fica fora do primeiro corte
Gatilho de repasse	Webhook válido <code>order.posted</code> , nunca o botão do produtor
Centro de distribuição/coleta	Fora do MVP

Situação atual do repositório

O projeto já possui `Order`, `OrderItem`, `Payment`, `Product`, `Producer`, checkout visual e criação básica de pedidos. Porém, ainda há lacunas que impedem a integração real:

1. `Product` não possui relação persistida com `Producer`; o café guarda apenas `producerName` como texto.
2. O pedido é plano e não possui subpedidos por produtor.
3. O frete é calculado no frontend por uma regra fixa de R\$ 15,90/grátis acima de R\$ 150.
4. O frontend simula pagamento e sucesso com `setTimeout`.
5. O backend aceita `shippingCents` enviado pelo navegador, o que permite adulteração de preço.
6. `PaymentProvider` e estados atuais não representam o ciclo do Asaas.
7. Não existem perfil logístico, embalagens, cotações persistidas, etiquetas, webhooks, razão financeira ou obrigações de repasse.
8. A API está concentrada em `server/src/index.ts`; as integrações devem nascer em módulos para não ampliar esse acoplamento.
9. O checkout ainda oferece débito e boleto, ambos fora do escopo aprovado.

10. O middleware JSON atual precisa preservar o corpo bruto do webhook do Melhor Envio para validar X-ME-Signature.

Arquitetura alvo

```
server/src/  
  modules/  
    checkout/  
      checkout.routes.ts  
      checkout.service.ts  
      checkout.schemas.ts  
    shipping/  
      shipping.types.ts  
      shipping.service.ts  
      packaging.service.ts  
      shipping.routes.ts  
      shipping.webhook.ts  
    providers/  
      shipping-provider.ts  
      melhor-envio.provider.ts  
  payments/  
    payment.types.ts  
    payment.service.ts  
    payment.routes.ts  
    payment.webhook.ts  
    ledger.service.ts  
    providers/  
      payment-provider.ts  
      asaas.provider.ts  
      payout-provider.ts  
      manual-payout.provider.ts  
  orders/  
    order.service.ts  
    order-status.service.ts  
  shared/  
    http-client.ts  
    money.ts  
    idempotency.ts  
    crypto.ts
```

Regras de negócio usam as interfaces internas. Nomes e payloads específicos do Melhor Envio e do Asaas ficam somente nos adaptadores.

Modelo de dados proposto

Catálogo e produtor

- Adicionar `producerId` em `Product` e migrar os cafés atuais pelo nome do produtor.
- `ProducerLogisticsProfile`: endereço de origem, dias de postagem, horário de corte, dias de preparação, serviços aceitos, capacidade de impressão e ponto de postagem.
- `PackagingTemplate`: código, dimensões, peso vazio, peso máximo, material e status.
- `ProductShippingProfile`: peso unitário e regras necessárias para selecionar a menor embalagem compatível.
- `ProducerPaymentProfile`: chave Pix criptografada, tipo, titular, documento, verificação, histórico e status.

Pedido e frete

- `SellerOrder`: subpedido ligado a `Order` e `Producer`, com subtotal, frete, status e elegibilidade de repasse.
- Associar cada `OrderItem` a um `SellerOrder`.
- `ShippingQuote`: produtor, serviço, transportadora, preço, prazo da transportadora, prazo prometido total, volumes, resposta bruta e expiração.

- **Shipment**: subpedido, cotação, ID no provedor, custo cotado/final, etiqueta, rastreo, status e datas operacionais.
- Permitir vários **Shipment** por **SellerOrder**, pois determinados serviços não aceitam múltiplos volumes na mesma etiqueta.
- **ShippingWebhookEvent**: chave de deduplicação, evento, etiqueta, validade da assinatura, hash do payload e datas de processamento.

Pagamento e razão financeira

- Evoluir **Payment**: **providerPaymentId**, **billingType**, valor bruto, valor líquido, taxa, **confirmedAt** e **receivedAt**.
- **SellerPayable**: uma obrigação imutável por subpedido/produtor, com subtotal, comissão, ajustes, valor devido, elegibilidade e pagamento.
- **Payout**: repasse manual, valor, chave mascarada, destinatário, identificador externo, comprovante e usuário responsável.
- **LedgerEntry**: lançamentos imutáveis de pagamento bruto, taxa, produtos, comissão, frete cobrado/comprado, variação, repasse, estorno e chargeback.
- **PaymentWebhookEvent**: ID do evento, tipo, cobrança, hash do payload e resultado do processamento.

Valores monetários permanecem em centavos. Peso deve ser normalizado em gramas no banco e convertido para quilogramas apenas no adaptador. Dimensões devem ter uma unidade única e explícita.

Estados e transições

Subpedido/envio

```
AWAITING_PAYMENT -> PAID -> PREPARING -> READY_FOR_LABEL
READY_FOR_LABEL -> LABEL_CREATED -> LABEL_PAID -> LABEL_GENERATED
LABEL_GENERATED -> AWAITING_POSTING -> POSTED -> IN_TRANSIT -> DELIVERED
```

Estados de exceção: **UNDELIVERED**, **PAUSED**, **SUSPENDED**, **CANCELLED** e **LABEL_EXPIRED**.

Somente eventos oficiais podem avançar **POSTED**, **IN_TRANSIT** e **DELIVERED**. A ação “Pedido embalado” pode chegar apenas a **READY_FOR_LABEL**.

Pagamento

```
PENDING -> RISK_ANALYSIS -> CONFIRMED -> RECEIVED
PENDING/CONFIRMED/RECEIVED -> REFUND_IN_PROGRESS -> REFUNDED
CONFIRMED/RECEIVED -> CHARGEBACK
```

- Pix libera preparação em **RECEIVED**.
- Cartão pode liberar preparação em **CONFIRMED**, conforme política de risco.
- Repasse exige simultaneamente **SellerOrder=POSTED**, obrigação não bloqueada e saldo disponível/conferido.

Contratos HTTP do produto

Cotação

POST /api/shipping/quotes

Entrada:

```
{
  "destinationPostalCode": "01001000",
  "items": [
    { "productId": "...", "quantity": 2, "metadata": { "weight": 250 } }
  ]
}
```

O servidor consulta produtos, preços, produtores e perfis logísticos no banco; agrupa por produtor; calcula volumes; consulta uma origem por grupo; filtra serviços aceitos; acrescenta agenda de postagem, preparo e margem; persiste cotações com validade curta.

Saída:

```
{
  "groups": [
    {
      "producerId": "...",
      "producerName": "...",
      "options": [
        {
          "quoteId": "...",
          "label": "ECONOMICO",
          "carrier": "...",
          "priceCents": 1890,
          "totalPromiseDays": 8,
          "expiresAt": "..."
        }
      ]
    }
  ],
  "expiresAt": "..."
}
```

Criação do pedido

POST /api/orders

O navegador envia itens, cliente, endereço e os `quoteId` escolhidos. O backend:

1. recarrega preços e estoque;
2. valida tenant, itens, destino, produtor e validade de cada cotação;
3. calcula todos os valores no servidor;
4. cria `Order`, `SellerOrder`, itens, `Payment`, `SellerPayable` e lançamentos iniciais na mesma transação;
5. rejeita valor de frete arbitrário enviado pelo cliente.

Cobrança

POST /api/orders/:orderId/payment

- Cria ou reutiliza cliente Asaas.
- Cria uma única cobrança com `externalReference=order.id`.
- Usa chave de idempotência interna para não criar duas cobranças.
- Retorna somente os dados necessários para Pix ou cartão.
- Dados completos de cartão nunca são persistidos nem enviados a logs.

Webhooks

- POST /api/webhooks/asaas: validar `asaas-access-token`, persistir evento e responder 2xx rapidamente.
- POST /api/webhooks/melhor-envio: validar o corpo bruto com HMAC-SHA256 e `X-ME-Signature`.
- Processamento idempotente por ID do evento quando disponível e por chave/hash determinístico como fallback.
- Não assumir ordem absoluta dos eventos; aplicar transições monotônicas e consultar o provedor quando houver divergência.

Operação administrativa

- GET /api/admin/seller-orders

- PATCH `/api/admin/seller-orders/:id/status` para ações operacionais permitidas.
- POST `/api/admin/seller-orders/:id/label`
- POST `/api/admin/shipments/:id/regenerate-label`
- GET `/api/admin/payouts/eligible`
- POST `/api/admin/payables/:id/manual-payout`

Sequência de implementação

Corte 0 — preparação do sandbox

- Criar contas independentes de sandbox no Melhor Envio e Asaas.
- Criar aplicativo no Melhor Envio e configurar webhook.
- Gerar chaves exclusivas de sandbox e armazená-las somente no backend.
- Expor os webhooks locais por uma URL HTTPS temporária.
- Cadastrar no seed um produtor com endereço logístico realista, agenda e embalagens de teste.

Resultado: credenciais válidas, endpoints alcançáveis e nenhum dado real de terceiros.

Corte 1 — fundação de domínio

- Criar migration com relação produto-produtor e entidades logísticas/financeiras.
- Migrar dados do seed e tornar todos os itens vendáveis atribuíveis a um produtor responsável.
- Extrair rotas do `server/src/index.ts` para os módulos novos.
- Adicionar validação de entrada e testes unitários de dinheiro, agrupamento, empacotamento e estados.

Resultado: domínio multi-produtor consistente, ainda sem chamadas externas.

Corte 2 — cotação real de frete

- Implementar `ShippingProvider` e `MelhorEnvioShippingProvider`.
- Implementar agrupamento, embalagens, filtros de serviço, prazo operacional e expiração.
- Trocar o frete fixo do checkout pelas opções persistidas retornadas pela API.
- Recalcular cotações expiradas e impedir que o frontend determine o preço.

Resultado: checkout calcula frete real por produtor no Melhor Envio Sandbox.

Corte 3 — pedido e cobrança Asaas

- Criar subpedidos e razão financeira de forma transacional.
- Implementar `AsaasPaymentProvider` para cliente, Pix e cartão.
- Remover débito e boleto da interface.
- Substituir o sucesso simulado por estados reais e tela de Pix/cartão.
- Implementar webhook do Asaas, autenticação, idempotência e atualização de pedido.

Resultado: uma cobrança real de sandbox controla a liberação de todos os subpedidos.

Corte 4 — etiqueta e rastreo

- Inserir frete no carrinho do Melhor Envio.
- Comprar, gerar e disponibilizar etiqueta após pagamento e preparo.

- Tratar geração assíncrona sem usar espera bloqueante.
- Implementar webhook assinado e estados até entrega.
- Exibir cada envio separadamente para administração e comprador.

Resultado: fluxo de etiqueta e rastreio completo no sandbox.

Corte 5 — repasse e conciliação

- Criar `SellerPayable` e fila de elegíveis após `order.posted`.
- Implementar `ManualPayoutProvider`, upload/referência do comprovante e dupla proteção contra duplicidade.
- Exibir demonstrativo de produto, comissão, taxas, frete e valor líquido.
- Bloquear/ajustar obrigações em estorno e chargeback.

Resultado: operação financeira auditável sem automatizar transferências.

Roteiro de homologação

1. Cotar um pedido de um produtor e validar preço, volume e promessa total.
2. Cotar um carrinho de dois produtores e confirmar duas origens, dois fretes e um total.
3. Expirar uma cotação e confirmar recálculo obrigatório.
4. Criar cobrança Pix, simular recebimento e validar liberação para preparo.
5. Criar cartão aprovado, recusado e em análise.
6. Reenviar o mesmo webhook e comprovar que nada é duplicado.
7. Gerar etiqueta, aguardar os estados automáticos do Melhor Envio Sandbox e validar `POSTED/DELIVERED`.
8. Tentar liberar repasse via botão do produtor e confirmar bloqueio.
9. Registrar repasse manual e impedir segunda liquidação da mesma obrigação.
10. Simular estorno e chargeback antes e depois da postagem.
11. Validar isolamento entre tenants em todas as consultas e mutações.
12. Executar uma postagem física real somente na homologação operacional fora do sandbox.

Guarda-corpos técnicos

- Nunca confiar em preço, produtor, frete, comissão ou total vindos do navegador.
- Nunca expor tokens de provedor em variáveis `VITE_*`.
- Não registrar chave Pix aberta, API keys ou dados completos de cartão em logs.
- Criptografar dados Pix em repouso e mascarar na interface.
- Toda escrita disparada por webhook ou chamada externa deve ser idempotente.
- Gravar resposta bruta dos provedores como auditoria, com remoção de dados sensíveis.
- Usar `outbox/job` persistido para operações externas que precisem de repetição; não depender de `setTimeout` em memória.
- Preservar `tenantId` e validar propriedade em todos os registros internos.
- Implementar reconciliação consultando Asaas e Melhor Envio para recuperar eventos perdidos.

Atualizações importantes da documentação oficial

- O Melhor Envio Sandbox usa <https://sandbox.melhorenvio.com.br>, exige User-Agent identificando a aplicação e contato, e possui contas/aplicativos separados da produção.
- O sandbox do Melhor Envio simula atualmente apenas Correios/Jadlog e avança postagem/entrega automaticamente em intervalos aproximados de 15 minutos.
- Ao inserir frete no carrinho, **products** passou a ser obrigatório para o fluxo de declaração de conteúdo/DC-e desde 6 de abril de 2026. A decisão fiscal precisa ser tratada no payload antes do lançamento.
- A geração de etiqueta é assíncrona; geração e impressão não devem ser tratadas como uma única resposta imediata.
- O Asaas Sandbox usa <https://api-sandbox.asaas.com/v3>, chave própria de homologação no header `access_token` e conta separada da produção.
- Webhooks do Asaas usam o token configurado no header `asaas-access-token`, seguem entrega pelo menos uma vez e devem responder 2xx rapidamente.

Dependências externas antes do primeiro teste integrado

- Credencial do aplicativo Melhor Envio Sandbox, secret e User-Agent de suporte.
- Credencial da conta Asaas Sandbox e token forte do webhook.
- URL HTTPS pública temporária para os dois webhooks.
- Endereço completo e agenda do produtor piloto.
- Pesos e medidas reais das caixas P, M e G.
- Definição fiscal para Nota Fiscal ou declaração de conteúdo/DC-e.
- Percentual de comissão e política de risco do cartão.

Sem essas definições ainda é possível implementar todo o domínio usando provedores falsos; as chamadas reais ficam ativadas por configuração quando as credenciais estiverem disponíveis.

Referências oficiais validadas

- Melhor Envio — Sandbox: <https://docs.melhorenvio.com.br/docs/sandbox>
- Melhor Envio — Cotação: <https://docs.melhorenvio.com.br/docs/cotacao-de-fretes>
- Melhor Envio — Carrinho: <https://docs.melhorenvio.com.br/reference/inserir-fretes-no-carrinho>
- Melhor Envio — Compra: <https://docs.melhorenvio.com.br/reference/compra-de-fretes-1>
- Melhor Envio — Webhooks: <https://docs.melhorenvio.com.br/docs/webhooks>
- Asaas — Sandbox: <https://docs.asaas.com/docs/sandbox>
- Asaas — Autenticação: <https://docs.asaas.com/docs/authentication-2>
- Asaas — Cobrança: <https://docs.asaas.com/reference/criar-nova-cobranca>
- Asaas — Webhooks: <https://docs.asaas.com/docs/about-webhooks>